Document Question Answering System (RAG)

Dataset:

Simple Beginner Dataset (Easiest)

Use your own PDFs:

- Notes
- Resume
- Research papers
- Books
- RAG is meant for **custom/private data**

Or Try this Hugging Face Dataset

Reference:Github Link

Overview

This project implements a Retrieval-Augmented Generation (RAG) system that answers questions based on custom documents.

Instead of relying only on a language model's internal knowledge, the system retrieves relevant information from documents and then generates answers grounded in that information. This improves factual accuracy and allows question answering over private or domain-specific data.

Objectives

- Understand the concept of Retrieval-Augmented Generation (RAG)
- Build a pipeline combining retrieval and generation
- Enable question answering over custom documents such as PDFs or text files
- Learn how modern AI systems work internally

Key Concepts

1. Retrieval
   Retrieval is responsible for finding the most relevant chunks of text from a document. It typically uses embeddings and vector similarity search.
2. Augmentation
   The retrieved content is added to the model's input to provide context for answering.

3. Generation
   A language model generates the final answer using the retrieved context, ensuring responses are grounded in actual data.

System Architecture

The pipeline consists of the following stages:

1. Document Ingestion
   Documents such as PDFs or text files are loaded and converted into raw text.
2. Text Chunking
   The text is split into smaller chunks to improve retrieval accuracy.
3. Embedding Creation
   Each chunk is converted into a vector representation capturing its semantic meaning.
4. Vector Database
   Embeddings are stored in a vector database for efficient similarity search.
5. Query Processing
   The user's question is converted into an embedding.
6. Context Retrieval
   The system retrieves the most relevant chunks from the database.
7. Answer Generation
   A language model generates an answer using the retrieved context.

Data

Input sources include:

- PDF documents
- Text files
- Notes or articles

These are typically unstructured and may contain domain-specific knowledge.

Components Used

- Embedding model for converting text into vectors
- Vector store for similarity search
- Language model for generating answers

Workflow

1. Load and preprocess documents
2. Split text into chunks
3. Convert chunks into embeddings
4. Store embeddings in a vector database
5. Accept user query
6. Retrieve relevant chunks
7. Generate answer using retrieved context

Example Flow

User Question:
“What is the main idea of the document?”

System Process:

- Retrieves relevant sections
- Provides them as context
- Generates a concise answer

Improvements & Experiments

- Use better chunking strategies
- Try different embedding models
- Improve retrieval using hybrid search (keyword + vector)
- Add re-ranking for better relevance
- Experiment with different language models

Key Learnings

- How RAG systems combine retrieval and generation
- Importance of retrieval in improving answer accuracy
- Working with embeddings and vector databases
- Handling unstructured text data
- Designing scalable AI pipelines

Conclusion

This project demonstrates how to build a system that can understand user queries, retrieve relevant information, and generate accurate answers.

RAG systems are widely used in chatbots, knowledge assistants, enterprise search systems, and AI-powered documentation tools.